

What a Denser Ternary Weight Format Is Actually Worth: a measurement on one CPU

Justus Theile

Independent researcher

`justus.theile@gmail.com`

Abstract

Ternary language models are stored in practice at two bits per weight, 26 % above the information content of a ternary symbol. Almost every mechanism for removing that excess is already published: packing five ternary values as the base-3 digits of a byte, recovering all five independently with reciprocal multiplication, contracting on the digit planes without materialising a weight, carrying the scale per tensor rather than per block, and the roofline criterion that decides when a denser code pays. We cite each to its owner in §1. What is missing from that literature is not another mechanism but a measurement: what the combination is worth end to end, on a real checkpoint, against the format that ships with it. We supply one. Integrated into `llama.cpp` as a distinct `ggml` type, the code reaches 1.60755 bits per weight on `BitNet-b1.58-2B-4T` — 4.7 % below the nearest published format on the ternary payload, 1.9 % of the file — with output bit-identical and perplexity agreeing with the two-bit baseline to six significant figures over 20,480 tokens. The profitability criterion is measured rather than assumed, one level down the memory hierarchy from where it was first stated, and both of its terms are taken from the part the kernel runs on; the crossing it predicts is observed between four and six threads for the isolated kernels, while the full model already wins at four — because the baseline `llama.cpp` dispatches is itself 1.6 times slower than the reference the ratio was taken against. Three results are new. A cheaper decode is worth $1.2254\times$ on the contraction and *nothing* for prompt processing, because a strip of width eight amortises the decode away entirely. The baseline kernel folds its accumulator ten times more often than its own bound requires, at a measured cost of 7.5 %. And 4.76 % of the checkpoint’s feed-forward neurons are identically zero, with the gate and up index sets coinciding in all thirty layers and none revived by an independent fine-tune. All figures are from a single AVX2 part without VNNI. Every one of them is recomputed by a script in the repository, but not every one from a clone alone: the throughput figures, the perplexity and the dead-neuron census read the checkpoint, which is not distributed there.

1 What is new here, and what is not

The format below places five ternary values in one byte as the digits of a base-3 integer, $3^5 = 243 < 256$, at 1.600 bits per weight. That construction is not ours and neither is most of what surrounds it. It is `llama.cpp`’s `TQ1_0` [1]; it is published with a correctness theorem as Spectra 1.1’s `TQ1` [2]; it appears a year before `TQ1_0` in statistical genomics, with the same bandwidth motivation and the same positional weights [3]; and it appears fifteen years before ternary language models in heuristic search [4]. Recovering all five digits *independently* from the packed byte, table-free, is NTRU’s reference implementation since 2019, at exactly these parameters [5], and is what `TQ1_0`’s own NEON path does. Contracting on the digit planes so that no weight is materialised is `TQ1_0`’s on both instruction sets. Carrying one scale per tensor instead of one per block was `block_q1_3`, on the pull request that became `TQ1_0`. The interleave that keeps the access pattern unchanged is `i2_s`’s at two bits and `TQ1_0`’s at 1.600. And the criterion for when a denser code pays is Equation 1 of Zukowski et al. [6], which already applies it as an adopt-or-reject rule for a codec.

We found each of these after building, not before, and the repository’s `CHANGELOG.md` records every one with how it was found. Stating them here rather than in a footnote is the point of the section: what follows is worth reading as a measurement, and a measurement is worth less when the reader has to work out for himself what part of it is a claim.

What is left. Four things, each small and each measured. A block length of $160 = 32 \times 5$, which takes no remainder byte where `TQ1_0`’s 64 does: 4,546,560 bytes, 0.38 % of the file. Separately, and worth no bytes at all, a scale riding in the baseline’s existing 32-byte tail, so the format is drop-in where the nearest scale-free predecessor needed a graph node and a loader change. The `AVX2` instantiation of the depth-one decode, which upstream has on `NEON` and, through a low-half 16-bit multiply, in its four-element tail — but not in its `AVX2` main path, where it names the question open in its own source. A column-blocked matrix–matrix kernel, which the nearest prior format has no equivalent of. And the measurements of §5, which are the reason for the paper.

2 The format

Let $c_i = w_i + 1 \in \{0, 1, 2\}$ be the shifted code of weight w_i . A block of 160 weights becomes 32 bytes,

$$b_j = \sum_{k=0}^4 c_{32k+j} 3^k, \quad j = 0, \dots, 31, \quad (1)$$

so $b_j \leq 2 \sum_k 3^k = 242$ and every byte is a legal base-3 word with thirteen codes to spare. The interleave $32k + j$ is the baseline’s: digit plane k addresses 32 *consecutive* activations, and the access pattern is therefore unchanged from the format replaced. Achieved density on the shipped checkpoint is 1.60755 bits per weight, the excess over 1.600 being tail padding at $K = 6,912$ plus one 32-byte record per tensor.

3 Decoding without a table

Because $b \leq 242$, all four quotients $\lfloor b/3^j \rfloor$ lie inside the exactness range of 16-bit reciprocal multipliers, so each can be taken *independently* from the original byte rather than from its predecessor: $\lfloor b/3 \rfloor$, $\lfloor b/9 \rfloor$, $\lfloor b/27 \rfloor$ and $\lfloor b/81 \rfloor$ come from one `vpmulhuw` each with multipliers 21,846, 7,282, 2,428 and 811. The digits follow by subtraction, $d_k = q_k - 3q_{k+1}$, and each plane is handed straight to `vpaddubsw` so that no weight is ever written to a register as a value. Dependency depth is one rather than five. Against the serial form this *encoding* admits the instruction count is identical — a chain $q_{j+1} = \text{mulhi}(q_j, 21,846)$ issues the same eight multiplies — so depth one costs nothing here; what it relieves is latency, which §5 shows is not the binding resource.

The accumulator bound. A `vpaddubsw` lane receives two products of at most 2×128 , so a 16-bit lane grows by at most 512 per plane and 2,560 per block, and folding to 32-bit every Φ blocks is safe iff $2,560 \Phi \leq 32,767$, hence $\Phi = 12$. The range $d \in \{0, 1, 2\}$ on which that rests is a consequence of $b \leq 242$ and not a property of a byte: the thirteen bytes 243...255 give $d_4 = 3$, which would bound a lane at 2,816 and overflow the fold. What the byte-wide subtraction needs is weaker and does hold for all 256 values — no digit subtraction borrows.

4 When a denser code pays

Let the baseline store at β_A bits and the candidate at $\beta_B < \beta_A$, and let $A_A > A_B$ be the arithmetic rates the two kernels attain with memory taken out of the measurement. With

$F = \beta_A/\beta_B$, $S = A_A/A_B$ and a surplus $\Sigma = A_A\beta_A/(8B)$ for memory bandwidth B , the denser code is faster exactly when

$$S < \Sigma. \quad (2)$$

The criterion is Zukowski’s; what is new is that both of its terms are measured here, and at a level of the hierarchy where they had not been. Two points decide whether the measurement is worth anything. A must be the *rate* and not the port ceiling: the baseline attains 63 % of its 130.9 GMAC/s ceiling and the packed kernel 67 % of its 50.5, and substituting ceilings would raise every Σ by half and reverse the prediction at two and four threads. And Σ is not a constant: at six threads it is 3.03 on an idle host and 1.65 to 2.38 across five runs under load. $S = 2.43$ is above every one of those loaded values and below the idle one, so the condition holds at six threads only while the machine is quiet.

5 Measurements

All figures are from one Ryzen 5 3600 (Zen 2, AVX2 and FMA, no VNNI, six cores).

Density and losslessness. The file falls from 1.10 GiB to 1.00 GiB; the ternary payload falls 4.7 % against the nearest published format. Output is bit-identical and perplexity agrees to six significant figures over 20,480 tokens of WikiText-2. A suite of 177,734 assertions runs from a clone with a compiler alone.

Throughput, and where it comes from. At four threads, llama-bench gives a median $1.119\times$ prompt and $1.133\times$ generation over five invocations, faster in five of five; the per-invocation spread runs to 1.253. The decomposition comes from one run that times both the matmuls and the wall clock of both arms:

	wall	matmuls	share	the rest
baseline	42.80 ms	21.18 ms	49.5 %	21.6 ms
this work	38.32 ms	16.70 ms	43.6 %	21.6 ms

The last column is identical between the arms *by construction*: only the packed arm carries a counter, and the baseline’s matmul time is defined as its wall clock minus the other arm’s non-matmul time. The table is a budget and not a cross-check — it shows that the untouched 21.6 ms is the larger half of a token, so a format acting on the smaller half is bounded by that. The matmul ratio itself is measured on both arms at one dispatch site over nine interleaved rounds: 1.257 median, the baseline slower in 9 of 9. The byte ratio 1.244, which shares no apparatus with a clock, agrees.

A cheaper decode, and the width at which it stops mattering. Deriving two of the five digits from q_3 with `vpshufb` — legal because $q_3 \leq 9$ is a nibble — removes one multiplier and six vector operations, $43 \rightarrow 37$, bit-identically. It is off by default; every other figure in this paper is from the build without it. It is worth $1.2254\times$ on the contraction, median of fifteen interleaved rounds with the untouched baseline kernel measured in the same binaries as a control, $p = 0.00098$. It is worth *nothing* for prompt processing, and the reason is structural: the matrix–matrix kernel decodes each block once and spreads it over the strip, so the gain is $1.1608\times$ at one activation column, $1.1363\times$ at four, and by width eight both arms saturate at the same rate. A decode improvement is a generation improvement only.

Two defects in the baseline kernel. Its dispatched path folds the int16 accumulator once per 128-weight block where its own operands license once per ten, costing a measured 7.5 % with identical perplexity. And its reference kernel folds every 32 blocks over four planes, up to $128 \times 508 = 65,024$ against a 32,767 ceiling; it is correct only because signed activations cancel, and removing the cancellation makes it disagree with exact arithmetic in 11,998 of 12,000 rows.

Dead neurons. 4.76 % of the checkpoint’s feed-forward neurons are identically zero. The dead index sets of the gate and up projections coincide in all thirty layers, and an independent fine-tune revives none of them. Removing them is not free: the sub-layer norm divides by the root mean over all 6,912 entries, so deletion rescales the output unless the divisor is pinned.

6 Limitations

One microarchitecture, and the decisive instruction is absent from it: with VPDPBUSD the contraction collapses into one instruction and the arithmetic ratio this paper turns on would change. A reviewer’s run on Intel hardware puts it 8.8 % higher, the direction predicted, on a host whose spread exceeds the effect. The absolute perplexity of this checkpoint is not reported: its GGUF needs two metadata fields supplied by hand, and the graph `llama.cpp` builds for its architecture applies SiLU where the checkpoint’s own `config.json` declares `relu2` — substituting it moves the perplexity from 96.8243 to 16.7482. Neither reaches the comparison: both arms run the same graph, so a wrong activation cancels exactly, which is why the identity result survives it. The 1.6 bits are the weights only; the tied f16 embedding is 55.3 % of the file and this format does not touch it.

Availability

Code, kernels, benchmarks, converters and every evidence file are at <https://github.com/purpleskull11/bitnet-t5b> under Apache 2.0. A clone builds and runs the suite with a compiler alone. `shufdig_ab.sh` reproduces the decode measurement, `mutation_test.sh` the correctness argument, and `tools/check_short_parity.py` the agreement between this paper and the long one. The long version of this paper, with the proofs, the mutation testing, the failed variants and the full correction history, is in the same repository.

References

- [1] compilade. *ggml-quants: ternary packing for TriLMs and BitNet b1.58*. llama.cpp pull request #8151, August 2024.
- [2] T. Vaidhya, A. Kaushal, V. Jain, F. Couture-Harpin, P. Shishodia, M. Behbahani, Y. Nevmyvaka, I. Rish. *Spectra 1.1: Scaling Laws and Efficient Inference for Ternary Language Models*. arXiv:2506.23025, 2025.
- [3] A. Freudenberg, J. Vandenplas, M. Schlather, T. Pook, R. Evans, J. ten Napel. *Accelerated matrix-vector multiplications for matrices involving genotype covariates with applications in genomic prediction*. Frontiers in Genetics 14:1220408, 2023.
- [4] T. M. Breyer, R. E. Korf. *1.6-Bit Pattern Databases*. Proceedings of the AAAI Conference on Artificial Intelligence 24(1):39–44, 2010.
- [5] J. M. Schanck et al. *NTRU reference implementation*, `ref-common/pack3.c`. <https://github.com/jschanck/ntru>, 2019.
- [6] M. Zukowski, S. Heman, N. Nes, P. Boncz. *Super-Scalar RAM-CPU Cache Compression*. ICDE, 2006.